Experiment 8: OO Design – Package, Component and Deployment Models

Tool used: StarUML 7.1.1
Diagram types: UML Package Diagram, UML Component Diagram, UML Deployment Diagram
Subject system: Orbital Sentinel – Satellite Telemetry Anomaly Detection System

Field	Value
Student name	Pushkar Kumar Mishra
Reg. number	24BPS1128
Course	BCSE301P
Date of submission	22nd October, 2026
StarUML project file	docs/uml/Exp8_Package_Component_Deployment.mdj
Source code	https://github.com/ItsPM25/Orbital_sentinel

1. Aim

To model the implementation and physical architecture of the Orbital Sentinel system: the packages of the code base and their dependencies (Package Diagram), the major software components and their interfaces (Component Diagram), and the hardware nodes, execution environments and deployed artefacts with the communication paths between them (Deployment Diagram).

2. System Overview

Orbital Sentinel is a web application with two parts. The frontend (Next.js 15 + React 19, port 3001) shows the alert dashboard, the health panel and a 3D globe. The backend (FastAPI on Python 3.12, port 8000) replays NASA SMAP/MSL telemetry through per-channel LSTM models, decides which windows are anomalous, explains and ranks them, stores the results in SQLite and pushes live updates to the browser with Server-Sent Events. Satellite positions are computed in the browser from TLEs that the backend fetches from CelesTrak.

3. Task 1 – Package and Component Model

3.1 Major functional modules

Module (package)	Folder	Function
web.app	web/app	Pages: login, alerts dashboard, globe, satellites, health panel, evaluation, administration.
web.components	web/components	Shared UI: app shell, page header, risk meter, shadcn/ui widgets.
web.lib	web/lib	Typed API client, SSE hook, request coalescing, SGP4 helpers, risk colours.
backend.app.api	backend/app/api	REST + SSE routers (auth, alerts, satellites, detections, ranking, stream, sessions, admin, evaluation, tle).
backend.app.schemas	backend/app/schemas	Pydantic request/response models (camelCase JSON).
backend.app.services	backend/app/services	sessions: running-session registry; events: publish/subscribe bus.
backend.app.pipeline	backend/app/pipeline	Dataset, ingest, LSTM, training, detection, lifecycle, monitoring session, fleet split.
backend.app.pipeline.explain	backend/app/pipeline/explain	Attribution, severity and risk ranking (the novelty layer).
backend.app.tracking	backend/app/tracking	CelesTrak client, TLE cache and bundled snapshot.
backend.app.evaluation	backend/app/evaluation	Metrics, baseline report, persistence-filter (REQ-11) reports.
backend.app.models	backend/app/models	SQLAlchemy tables, one class per ER entity.
backend.app.core	backend/app/core	db (SQLite, WAL), security (argon2, signed cookie, roles), settings (OS_* variables).

ml.spike	ml/spike	Phase 1 model-selection spike (throwaway prototype).

3.2 Package Diagram

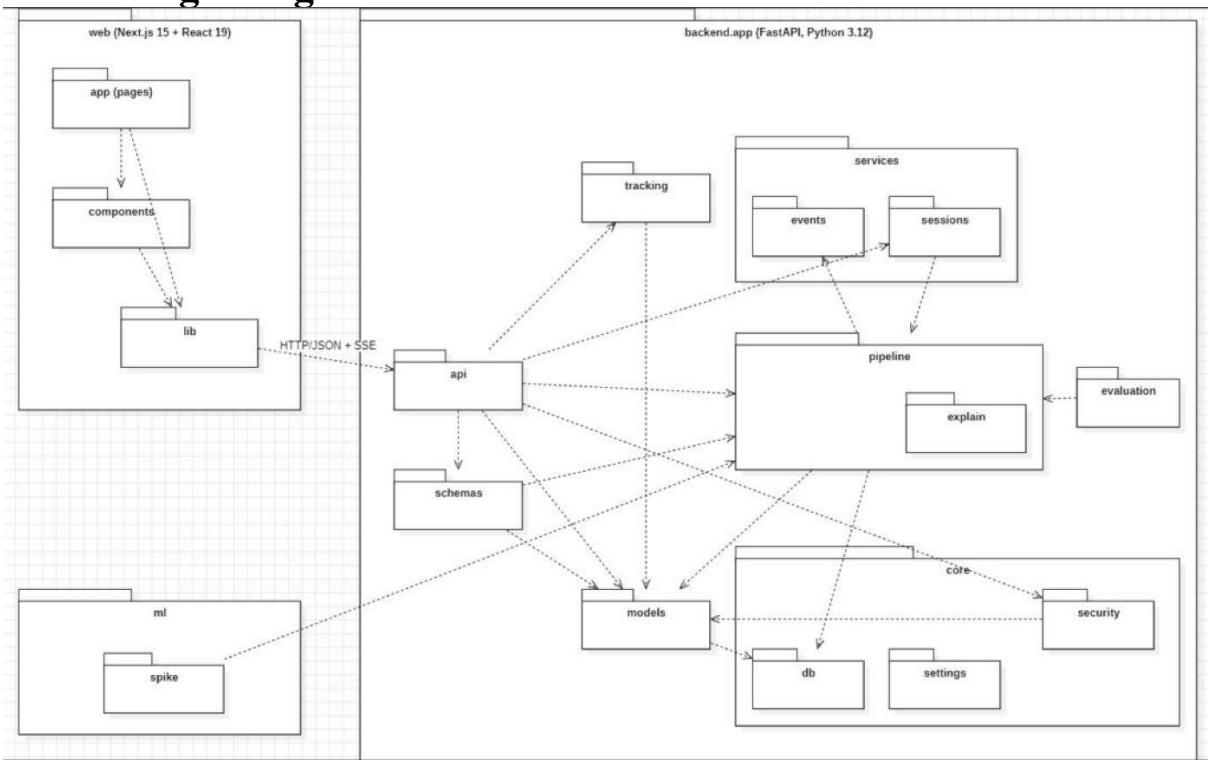


Figure 1: Package Diagram – Orbital Sentinel

Each dashed arrow is a dependency: the source package imports the target package. The table lists every dependency found in the code; the imports of core.settings and core.db by several packages are left off the drawing to keep it readable.

Package	Depends on	Why
web.app	web.components, web.lib	Pages are built from shared components and call the API client.
web.components	web.lib	Components use the risk helpers and formatters.
web.lib	backend.app.api (HTTP/JSON + SSE)	The only link between frontend and backend is HTTP through the /api rewrite.
api	schemas, services.sessions, services.events, pipeline, tracking, models, core.security/db/settings	Routers validate input, call services and return schemas.
schemas	models, pipeline.explain	Response models are built from ORM rows and ranking results.
services.sessions	pipeline, models, core	Starts MonitoringSession tasks.
pipeline	services.events, models, core.db, core.settings	Writes detections and alerts, and publishes live events.
evaluation	pipeline, core.settings	Re-uses the live scoring code to measure accuracy.
tracking	models	Caches fetched TLEs on the satellite rows.
models	core.db	Tables extend the declarative Base.
core.security	models	Loads the signed-in User.
ml.spike	pipeline (dataset)	The spike loads the same dataset files.

3.3 Component Diagram

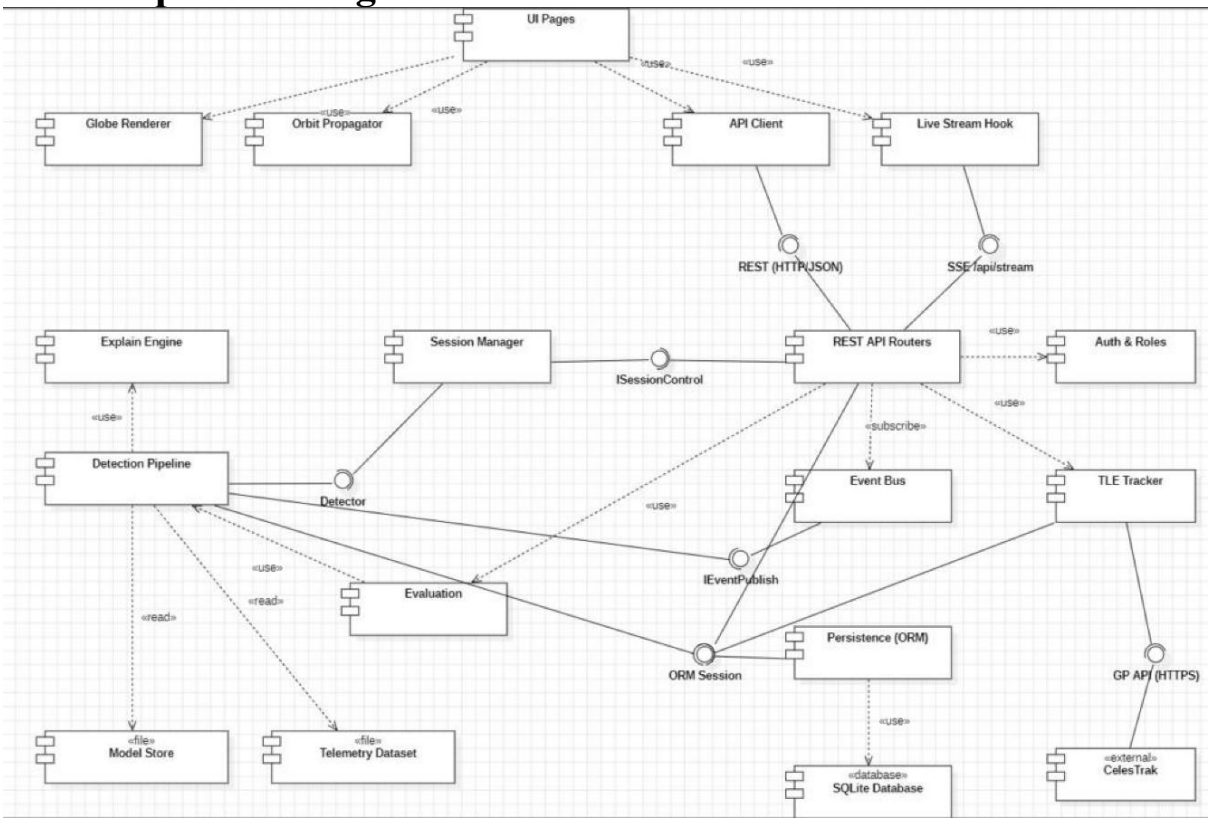


Figure 2: Component Diagram – Orbital Sentinel

Component	Provides	Requires / uses	Realised by
UI Pages	–	Globe Renderer, Orbit Propagator, API Client, Live Stream Hook	web/app
Globe Renderer	–	–	globe.gl 2.46 + three.js 0.186
Orbit Propagator	–	–	web/lib/sgp4.ts (satellite.js)
API Client	–	REST (HTTP/JSON)	web/lib/api.ts
Live Stream Hook	–	SSE /api/stream	web/lib/useStream.ts
REST API Routers	REST (HTTP/JSON), SSE /api/stream	ISessionControl, ORM Session, Auth & Roles, Event Bus, TLE Tracker, Evaluation	backend/app/api
Auth & Roles	–	–	core/security.py
Session Manager	ISessionControl	Detector	services/sessions.py
Detection Pipeline	Detector	IEventPublish, ORM Session, Explain Engine, Model Store, Telemetry Dataset	pipeline/
Explain Engine	–	–	pipeline/explain/
Event Bus	IEventPublish	–	services/events.py
TLE Tracker	–	GP API (HTTPS), ORM Session	tracking/celestrak.py
Evaluation	–	Detection Pipeline	evaluation/
Persistence (ORM)	ORM Session	SQLite Database	models/ + core/db.py
SQLite Database «database»	–	–	backend/orbital.db (WAL)
Model Store, Telemetry Dataset «file»	–	–	models/, data/
CelesTrak «external»	GP API (HTTPS)	–	celestrak.org

Provided interfaces are drawn as lollipops (realization) and required interfaces as sockets (dependency). Interfaces sit where one part of the system could be replaced: the Detector interface lets a different model replace the LSTM, and the ORM Session lets Postgres replace SQLite (OS_DATABASE_URL).

4. Task 2 – Deployment Model

4.1 Nodes and execution environments

Node	Stereotype	Contains
Analyst Workstation	«device»	Web Browser (Chrome / Firefox) «executionEnvironment»
Application Host (Windows 11 laptop: i5-13420H, 12 logical CPUs, 16 GB RAM)	«device»	Node.js 22 runtime, Python 3.12 venv (Uvicorn), Local file system «executionEnvironment»
CelesTrak Server	«device» (external)	GP API web service
Kaggle	«device» (external, one-off)	Dataset mirror
GitHub	«device» (external)	Repository and Actions CI runners

4.2 Deployed artifacts

Artifact	Deployed on	Notes
Next.js client bundle (JS), globe.gl + three.js, satellite.js (SGP4)	Web Browser	Downloaded from the Next.js server; SGP4 runs every frame in the browser.
web (Next.js server :3001)	Node.js 22 runtime	Production build (next start); rewrites /api/* to port 8000.
backend.app (FastAPI :8000)	Python 3.12 venv	REST + SSE, monitoring sessions.
PyTorch 2.14 CPU runtime	Python 3.12 venv	Inference capped at 2 threads per process.
MDL-LSTM-100-80 (81 .pt files)	Python 3.12 venv (models/ folder)	One LSTM per labelled channel, about 29 MB.
orbital.db (SQLite, WAL)	Local file system	Config (D1) and log (D2) data stores.
data/ SMAP-MSL .npy (248 MB)	Local file system	NASA expert-labelled telemetry, replayed.
GP API (gp.php)	CelesTrak Server	TLE source.
nasa-anomaly-detection-dataset-smap-msl	Kaggle	Downloaded once by scripts/get_data.*
Actions runners (Ubuntu, Windows)	GitHub	ci.yml: ruff, pytest, tsc, next build.

4.3 Communication paths

Between	Protocol	Purpose
Analyst Workstation ↔ Application Host	HTTP :3001 (JSON + SSE)	Pages, API calls and live events.
Node.js runtime ↔ Python venv	HTTP localhost:8000	Next.js forwards /api/* to FastAPI (same origin for the browser).
Application Host ↔ CelesTrak Server	HTTPS	TLE download, cached 2 h, 5 s timeout, 10 min back-off.
Application Host ↔ Kaggle	HTTPS	One-off dataset download.
Application Host ↔ GitHub	HTTPS	git push; CI runs on every push.

4.4 Deployment Diagram

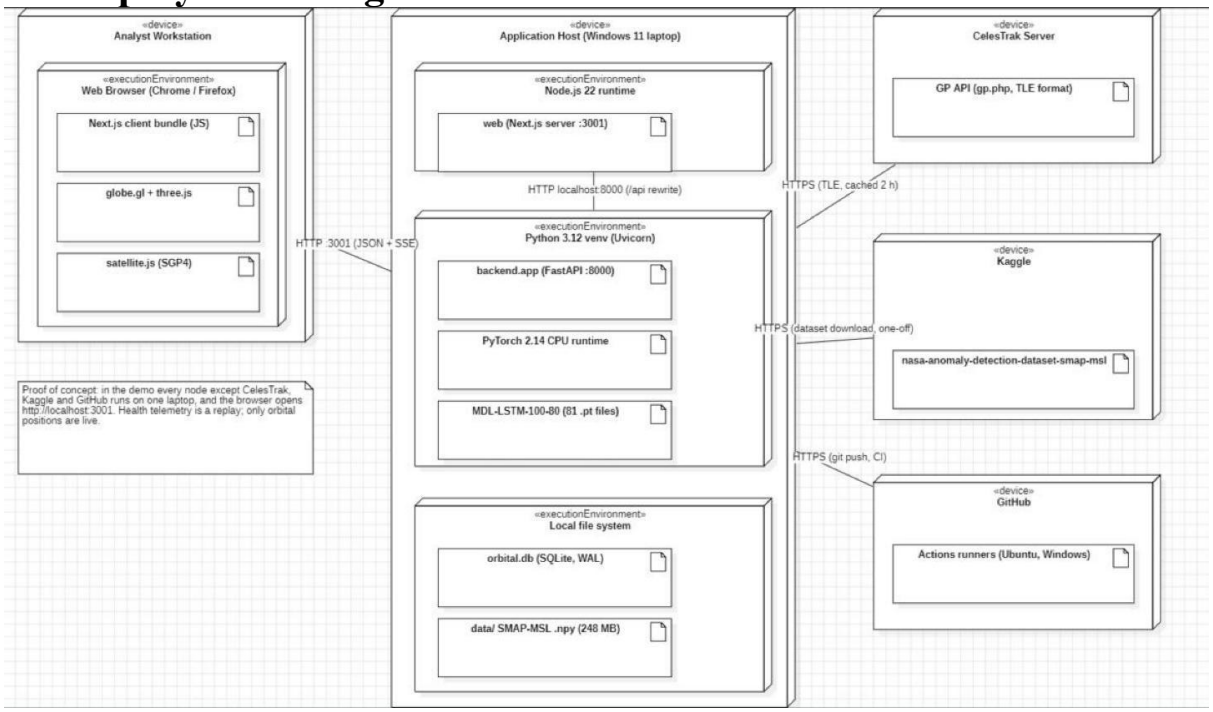


Figure 3: Deployment Diagram – Orbital Sentinel

The note on the diagram records the honest deployment: this is a proof of concept, so in the demo the workstation and the application host are the same laptop and the browser opens http://localhost:3001.

5. Procedure Followed

1. Opened StarUML and created a new project (saved as docs/uml/Exp8_Package_Component_Deployment.mdj).

2. Created a Package Diagram; added the web, backend.app and ml packages with their sub-packages, arranged in layers.

3. Connected packages with dependency relationships taken from the actual import statements in the code.

4. Created a Component Diagram; added the browser, backend, data and external components.

5. Added provided interfaces (lollipop) and required interfaces (socket), plus «use» and «read» dependencies.

6. Created a Deployment Diagram; added the device nodes for the workstation, the application host and the external services.

7. Placed the execution environments inside the nodes and the artifacts inside the execution environments.

8. Connected the nodes with communication paths labelled with their protocol.

9. Checked notation and relationships, re-arranged elements so that no line crosses a box, and saved the project.

Element creation was scripted through StarUML's extension API (source in docs/uml/staruml-extension/main.js) so that every name matches the code exactly; the layout was then checked and adjusted in StarUML.

6. Design Notes

Layering. Dependencies point downwards: api → services/pipeline → models → core. The browser reaches the backend only through the HTTP interface, never through shared code.

Cycles made visible and narrowed. At package level there are two import cycles (pipeline ↔ services and models ↔ core). Drawing the sub-packages shows that each is really between different modules (pipeline → services.events while services.sessions → pipeline; models → core.db while core.security → models), so no single module depends on itself. Moving events into its own top-level package would remove the cycle entirely.
main.py is not drawn. backend/app/main.py is a module (the FastAPI entry point that wires the routers), not a package, so it is not shown on the package diagram.
Replaceable components. The Detector and ORM Session interfaces are the planned replacement points (a different model; Postgres instead of SQLite).

7. Full-Screen Screenshots (with timestamp)

Screenshots of StarUML with the project open; the Windows taskbar clock shows the date and time.

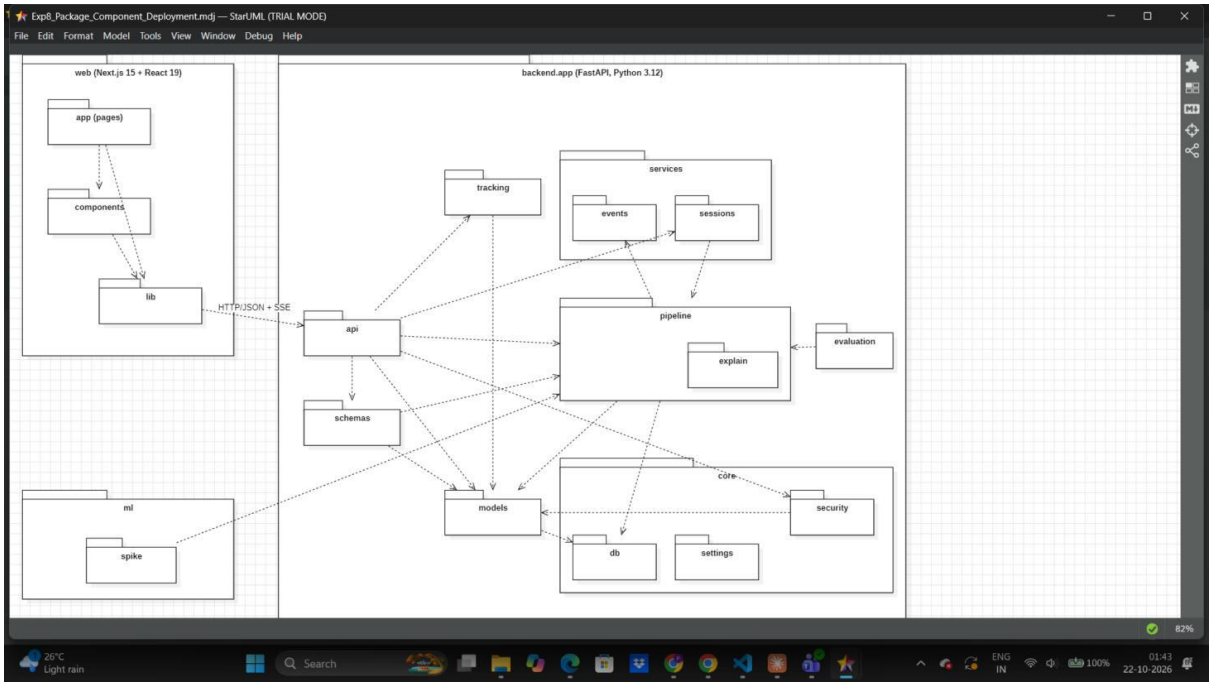


Figure 4: Package Diagram open in StarUML, with system timestamp

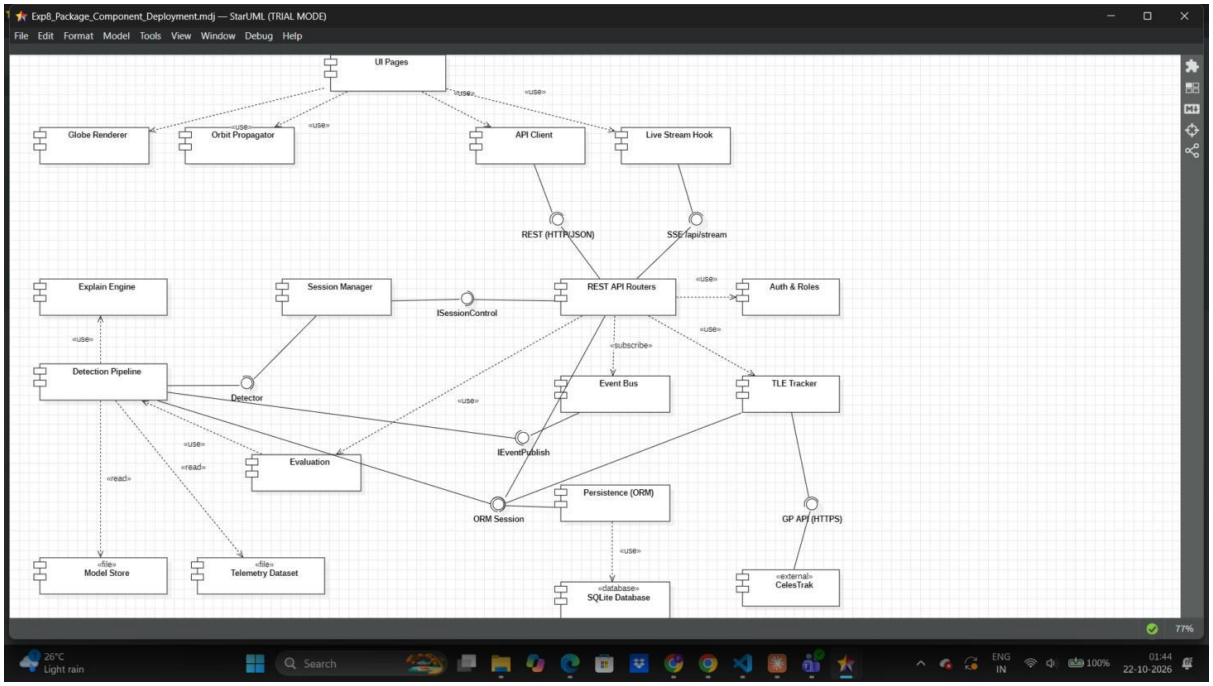


Figure 5: Component Diagram open in StarUML, with system timestamp

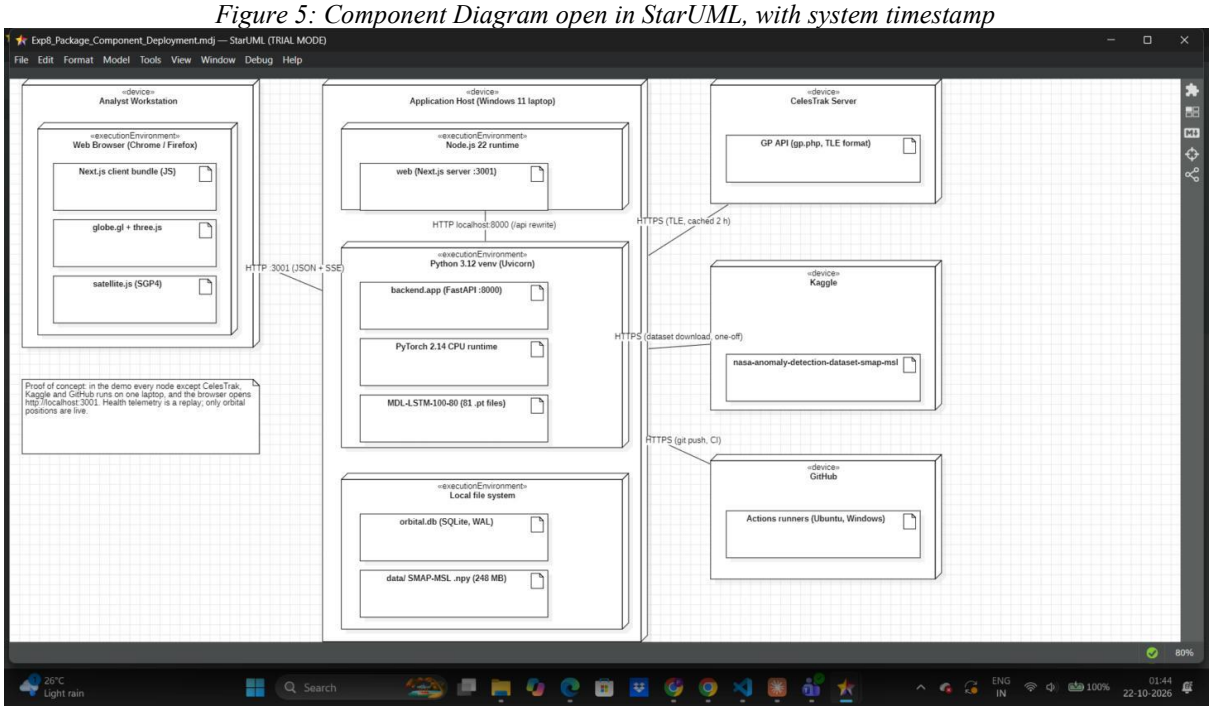


Figure 6: Deployment Diagram open in StarUML, with system timestamp

8. Submission Checklist

☑ Major functional modules identified (Section 3.1)

☑ Package Diagram with packages and dependencies (Figure 1)

☑ Component Diagram with components, interfaces and dependencies (Figure 2)

☑ Hardware/environment nodes identified (Section 4.1)

☑ Deployment Diagram with devices, execution environments, deployed artifacts and communication paths (Figure 3)

☑ UML notation and relationships checked

☑ StarUML project saved (docs/uml/Exp8_Package_Component_Deployment.mdj)

☑ Timestamp screenshots attached (Section 7)

9. Result

The Package, Component and Deployment models of the Orbital Sentinel system were designed in StarUML. The package diagram shows 3 top-level packages and 19 sub-packages with their real import dependencies; the component diagram shows 18 components and 7 interfaces; the deployment diagram shows 5 device nodes, 4 execution environments, 12 deployed artifacts and 5 communication paths. Together they describe how the code is organised, how the running parts talk to each other, and where each part runs.